

Format Bank

COMP336, Analysis of Algorithms - تحليل الخوارزميات

☐ Problem Solving	2
☐ Dynamic Programming	5
☐ Huffman Coding	12
☐ Graphs	16
☐ Backtracking	22
☐ Parallel Processing.....	24

❏ Problem Solving

Q(PS): Assume you are given a number X and two sorted lists of n number each. $A_1 \leq A_2 \leq \dots \leq A_n$ and $B_1 \leq B_2 \leq \dots \leq B_n$. Design a linear (i.e. $O(n)$) time algorithm to determine how many pairs (i, j) there are such that $A_i + B_j \leq X$. Justify the correctness and running time of your algorithm.

```
function count_pairs(A, B, X):
    n = length of A
    i = 0
    j = n - 1
    count = 0

    while i < n and j >= 0:
        if A[i] + B[j] <= X:
            count = count + (j + 1)
            i = i + 1
        else:
            j = j - 1

    return count
```

The algorithm correctly counts all pairs (i, j) where $A_i + B_j \leq X$ by leveraging the sorted nature of the lists. When $A_i + B_j \leq X$, all elements before B_j will also satisfy the condition with A_i , so we can increment the count accordingly.

Q(PS): Find missing number. An array of n elements contains all but one of the integers from 1 to $n + 1$

1. Give the best algorithm you can for determining which number is missing if the array is sorted, and analyze its asymptotic worst-case running time.
2. Give the best algorithm you can for determining which number is missing if the array is not sorted, and analyze its asymptotic worst-case running time.

Array is sorted:

```
function findMissing(array, n):
    left = 0
    right = n - 1

    while left <= right:
        mid = left + (right - left) / 2

        if array[mid] == mid + 1:
            left = mid + 1
        else:
            right = mid - 1

    return left + 1
```

Using binary search to determine if the missing number is in the left or right half of the array the worst-case will be $O(\log n)$.

Array is not sorted

```
function findMissing(array, n):
    expectedSum = (n + 1) * (n + 2) / 2
    sum = 0

    for num in array:
        sum += num

    return expectedSum - sum
```

Using the mathematical approach to find the missing number. Calculate the expected sum of numbers from 1 to $n + 1$ and subtract the sum of the elements in the array. This will make the worst-case $O(n)$.

Q(PS): An array $A[1, \dots, 2n+1]$ is wiggly if $A[1] \leq A[2] \geq A[3] \leq A[4] \geq \dots \leq A[2n] \geq A[2n+1]$. Given an unsorted array $B[1, \dots, 2n+1]$ of real numbers, describe an efficient algorithm that outputs a permutation $A[1 \dots 2n+1]$ of B such that A is a wiggly array.

```
function makeWiggly(B, n) begin
  A = B
  for i from 1 to 2n do
    if i is even then
      if A[i] < A[i+1] then
        swap(A[i], A[i+1])
      endif
    else
      if A[i] > A[i+1] then
        swap(A[i], A[i+1])
      endif
    endif
  endfor
  return A
end
```

Which has a time complexity of $O(n)$

□ Dynamic Programming

Q(DP): Given 5 matrices A(3x5), B(5x4), C(4x6), D(6x7), and E(7x3). Using the dynamic programming technique, we arrived at the following main last entry (1,5). What is the missing cell? And what is the matrix grouping that gives the minimum cost?

	A (1)	B (2)	C (3)	D (4)	E (5)
A (1)	0	60 1	132 2	258 3	?
B (2)		0	120 2	308 2	258 2
C (3)			0	168 3	198 3
D (4)				0	126 4
E (5)					0

? = 294 | 2

Group = (AB)(C(DE))

Q(DP): How can you optimize the space complexity of the Knapsack Problem from $O(nm)$ to $O(n)$. Write the algorithm for it.

```
function knapsack(weights, p, capacity):
    n = length(weights)
    c = array of size (capacity + 1) initialized to 0
    for i = 0 to n-1
        for w = weights[i] to capacity:
            c[w] = max(c[w], c[w - weights[i]] + p[i])

    return c[capacity]
```

Now using one dimensional array for the dynamic table. Which will result in $O(n)$ space complexity.

Q(DP): Give an algorithm using dynamic programming to determine how many distinct ways there are to give a cent in change using any coins from among pennies (1), nickels (5), dimes (10), and quarters (25). For example, there are 6 ways to give 16 cents change:

- A dime, a nickel, and a penny;
- A dime and 6 pennies;
- 3 nickels and a penny;
- 2 nickels and 6 pennies;
- One nickel and 11 pennies;
- And 16 pennies.

Demonstrate your solution by showing a step-by-step solution for 12 cents change.

$$table[i][j] = \begin{cases} 1 & , j = 0 \\ 0 & , i = 0 \text{ and } j > 0 \\ table[i-1][j] & , coins[i] > j \\ table[i-1][j] + table[i][j - coins[i]] & , \text{otherwise} \end{cases}$$

<i>i</i>		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0		1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	P (1)	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	P (5)	1	1	1	1	1	2	2	2	2	2	3	3	3	3	3	4	4
3	P (10)	1	1	1	1	1	2	2	2	2	2	4	4	4	4	4	6	6
4	P (25)	1	1	1	1	1	2	2	2	2	2	4	4	4	4	4	6	6

```
function countWays(amount, coins):
    n = length(coins)
    table = array of size (n+1) x (amount+1) initialized to 0

    for i from 0 to n:
        table[i][0] = 1

    for i from 1 to n:
        for j from 1 to amount:
            if coins[i] > j:
                table[i][j] = table[i-1][j]
            else:
                table[i][j] = table[i-1][j] + table[i][j - coins[i]]

    return table[n][amount]
```


Q(DP): Balanced Partition, you have a set of n integers each in the range $0 \dots k$, partition these integers into two subsets such that $S1 - S2 = 0$, there $S1$ and $S2$ denote the sums of the elements in each of the subsets. (The sum of elements in both subsets is same).

Hint: $part[i][j] = \text{true}$ if a subset of $\{arr[0], arr[1], \dots, arr[j-1]\}$ has sum equal to i . otherwise false.

If Input: `int arr[] = {3, 1, 1, 2, 2, 1};`

Complete the following table:

	{}	{3}	{3, 1}	{3, 1, 1}	{3, 1, 1, 2}	{3, 1, 1, 2, 2}	{3, 1, 1, 2, 2, 1}
0	True	True	True	True	True	True	True
1	False	False	True	True	True	True	True
2	False	False	False	True	True	True	True
3	False	True	True	True	True	True	True
4	False	False	True	True	True	True	True
5	False	False	False	True	True	True	True

Complete the following code:

```
// Returns true if arr[] can be partitioned into two subsets of equal sum
boolean findPartition(int arr[], int n) {
    int sum = 0;
    int i, j;
    // Calculate sum of all elements
    for (i = 0; i < n; i++) sum += arr[i];

    if (sum % 2 != 0) return false;

    boolean part[][] = new boolean[sum / 2 + 1][n + 1];

    // Initialize top row as true
    for (i = 0; i <= n; i++) part[0][i] = true;

    // Initialize leftmost column, except part[0][0], as false
    for (i = 1; i <= sum / 2; i++) part[i][0] = false;

    // Fill the partition table [FILL HERE ONLY]
    for (i = 1; i <= sum / 2; i++) {
        for (j = 1; j <= n; j++) {
            part[i][j] = part[i][j - 1];
            if (i >= arr[j - 1])
                part[i][j] = part[i][j] || part[i - arr[j - 1]][j - 1];
        }
    }

    return part[sum / 2][n];
}
```

Q(DP): Given a list of n integers, $v_1, \dots, v_n$, the product-sum is the largest sum that can be formed by multiplying adjacent elements in the list. Each element can be matched with at most one of its neighbors. For example, given the list 1,2,3,1, the product sum is $8=1+(2 \times 3)+1$.

Give the optimization formula for computing the product-sum of the first j elements.

$$ps[j] = \begin{cases} v_j & , \quad j = 0 \\ v_j \times v_{j-1} & , \quad j = 1 \\ \max(ps[j-1] + v_j, ps[j-2] + v_j \times v_{j-1}), & \text{otherwise} \end{cases}$$

Compute the product-sum of the list 1, 4, 3, 2, 3, 4, 2.

1 (i = 0)	4 (i = 1)	3 (i = 2)	2 (i = 3)	3 (i = 4)	4 (i = 5)	2 (i = 6)
1	4	13	15	18	22	24

Q(DP): Give an algorithm that makes change for an amount C using the fewest number of coins possible, given n types of coins where one of the coins has a value of 1.

$$nc[i][j] = \begin{cases} 0 & , \quad j = 0 \\ nc[i-1][j] & , \quad \text{coins}[i] > j \\ 1 + ns[i][j - \text{coins}[i]], & \text{coins}[i] \leq j \end{cases}$$

```
function coinChange(coins, C):
    n = length(coins)
    nc = array of size (n+1) x (C+1)

    for i from 0 to n:
        nc[i][0] = 0

    for i from 1 to n:
        for j from 1 to C:
            if coins[i-1] > j:
                nc[i][j] = nc[i-1][j]
            else:
                nc[i][j] = nc[i][j - coins[i-1]] + 1
    return nc[n][amount]
```

Note: the space complexity can be optimized even more (by using 1D dynamic array)

Q(DP): Given a string 'S', find the length of the Longest Palindromic Subsequence in it. The Longest Palindromic Subsequence (LPS) is the problem of finding a maximum-length subsequence of a given string that is also a Palindrome. Show the table for S = "BBABCBCAB"

$$lp[i][j] = \begin{cases} 1 & , \quad i = j \\ \max(lp[i+1][j], lp[i][j-1]), & S[i] \neq S[j] \\ 2 + lp[i+1][j-1] & , \quad S[i] = S[j] \end{cases}$$

	B	B	A	B	C	B	C	A	B
B	1	2	2	3	3	5	5	5	7
B		1	1	3	3	3	3	5	7
A			1	1	1	3	3	5	5
B				1	1	3	3	3	5
C					1	1	3	3	3
B						1	1	1	3
C							1	1	1
A								1	1
B									1

```
function lps(s):
    n = length(s)
    lp = array[n][n]
    for i = 0 to n-1:
        lp[i][i] = 1

    for i = n-2 down to 0:
        for j = i+1 to n-1:
            if s[i] == s[j]:
                lp[i][j] = lp[i+1][j-1] + 2
            else:
                lp[i][j] = max(lp[i+1][j], lp[i][j-1])

    return lp[0][n-1]
```

Note: the space complexity can be optimized even more (by using 1D dynamic array)

Q(DP): Max profit travelling

You operate a costume business that has two offices, one in Jerusalem (**JR**) and one in Amman (**AM**). Each week, you need to decide whether you want to work in **JR** or **AM** office. Depending on the week, your business makes more profit by having you at one office or the other. You are given a table of weekly profits, based on your location. Here is an example:

WEEK	1	2	3	4	5
JR	\$400	\$100	\$200	\$50	\$1100
AM	\$210	\$900	\$100	\$1500	\$20

Clearly, you would prefer to work at the location where you get the greater profit but here is the catch. It cost \$1000 to fly from one office to the other. (For example, if you do the first job in **JR**, the next three in **AM**, and the last in **JR**, the total profit will be: $400 - 1000 + (900 + 100 + 1500) - 1000 + 1100 = 2000$)

You are given two lists, $JR[1..n]$ and $AM[1..N]$, where $JR[i]$ is the profit for spending week i in **JR**, same for $AM[i]$. Present an efficient algorithm, which given these two arrays, determines your maximum overall profit. You must start and end in JR, but you may travel back and forth any number of times. Briefly justify your algorithm's correctness and derive its running time.

Hint: $O(n)$ time is possible using dynamic programming. It suffices to just the recursive rule. You will need to find a way to keep track of where you were the previous week.

Answer:

```
function calc(jr, am, n) begin
  p = new Array[n][2]
  p[0][0] = jr[0]    // profit[i][0] tracks jr profit
  p[0][1] = INT_MIN // profit[i][1] tracks am profit

  for i from 1 to n-1 begin
    p[i][0] = max(p[i-1][0] + jr[i], p[i-1][1] + jr[i] - 1000)
    p[i][1] = max(p[i-1][1] + am[i], p[i-1][0] + am[i] - 1000)
  endfor

  return dp[n-1][0]
end
```

The result table:

	1	2	3	4	5
JR [J = 0]	400	500	700	750	2000
AM [J = 1]	$-\infty$	300	400	1900	1920

Q(DP): You are going on a long trip. You start on the road at mile post 0. Along the way there are n hotels, at mile posts $a_1 < a_2 < \dots < a_n$, where each a_i is measured from the starting point. The only places you are allowed to stop are at these hotels, but you can choose which of the hotels you stop at. You must stop at the final hotel (at distance a_n), which is your destination. You'd ideally like to travel 200 miles a day, but this may not be possible (depending on the spacing of the hotels). If you travel x miles during a day, the penalty for that day is $(200 - x)^2$. You want to plan your trip so as to minimize the total penalty that is, the sum, over all travel days, of the daily penalties. Give an efficient algorithm that determines the optimal sequence of hotels at which to stop.

$a = [100, 250, 400, 600, 750, 900]$

0 (0)	1 (100)	2 (250)	3 (400)	4 (600)	5 (750)	6 (900)
0	10000	2500	5000	5000	7500	10000

$$pal[i] = \begin{cases} 0, & i = 0 \\ \min_{j \in [0, i-1]} (a[j] + (200 - a[i] - a[j])^2), & otherwise \end{cases}$$

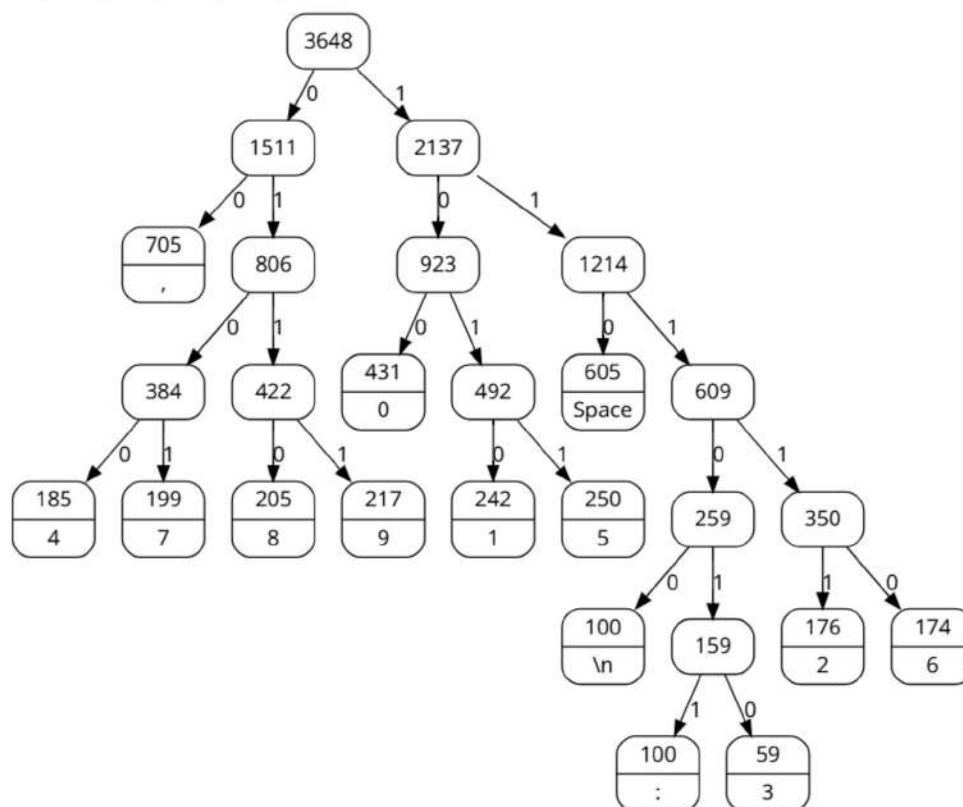
»Important problems you need to solve: «

Search for $\mathcal{O}$: [Top 20 Dynamic Programming Interview Questions - GeeksforGeeks](https://www.geeksforgeeks.org/top-20-dynamic-programming-interview-questions/)

(<https://www.geeksforgeeks.org/top-20-dynamic-programming-interview-questions/>)

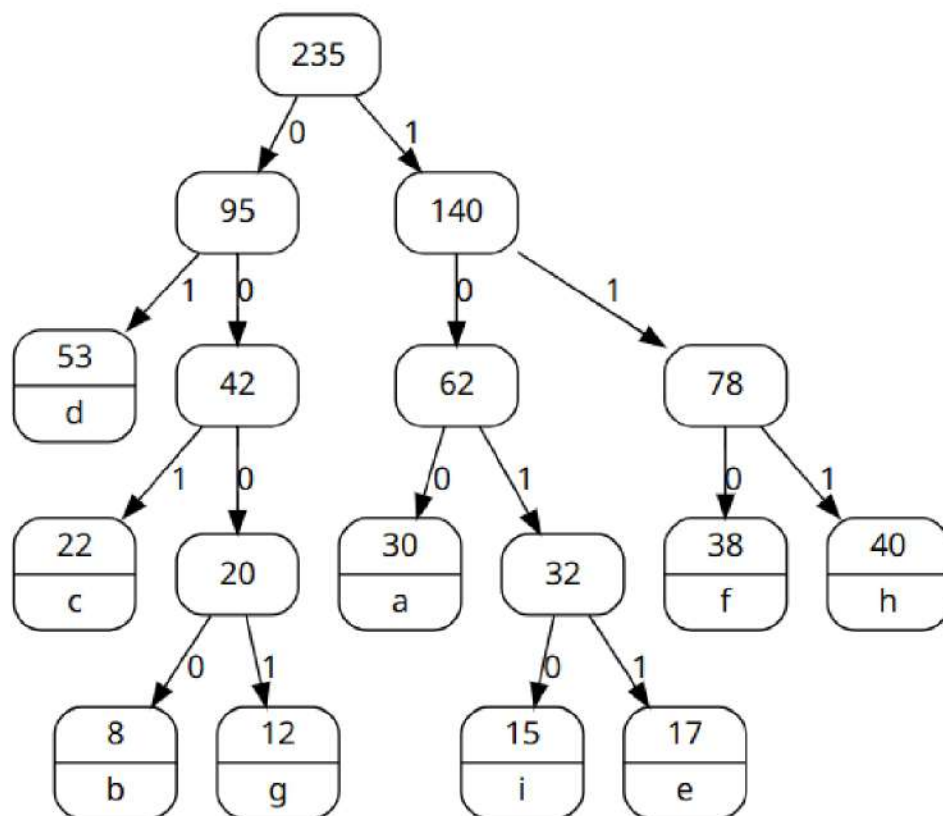
□ Huffman Coding

Q(HUF): A file contains the following: colon (100), space (605), newline (100), comma (705), 0 (431), 1 (242), 2 (176), 3 (59), 4 (185), 5 (250), 6 (174), 7 (199), 8 (205), 9 (217). Construct the Huffman Code.



Char	Freq.	Code	Size
:	100	111011	6
\s	605	110	3
\\n	100	11100	5
,	705	00	2
0	431	100	3
1	242	1010	4
2	176	11111	5
3	59	111010	6
4	185	0100	4
5	250	1011	4
6	174	11110	5
7	199	0101	4
8	205	0110	4
9	217	0111	4

Q(HUF): A file contains the following frequencies for characters: a (30), b (8), c (22), d (53), e (17), f (38), g (12), h (40), i (15). Construct the Huffman Code for these characters.



- What is the total size in bits if we used ASCII?
 $235 \times 8 = 1880$
- What is the total size in bits if we used fixed length?
 $235 \times 3 = 705$
- What is the total size in bits if we used Huffman?
 $(30 \times 3) + (8 \times 4) + (22 \times 3) + (53 \times 2) + (17 \times 4) + (38 \times 3) + (12 \times 4) + (40 \times 3) + (15 \times 4) = 704$

Char	Freq.	Code	Size
a	30	100	3
b	8	0000	4
c	22	001	3
d	53	01	2
e	17	1011	4
f	38	110	3
g	12	0001	4
h	40	111	3
i	15	1010	4

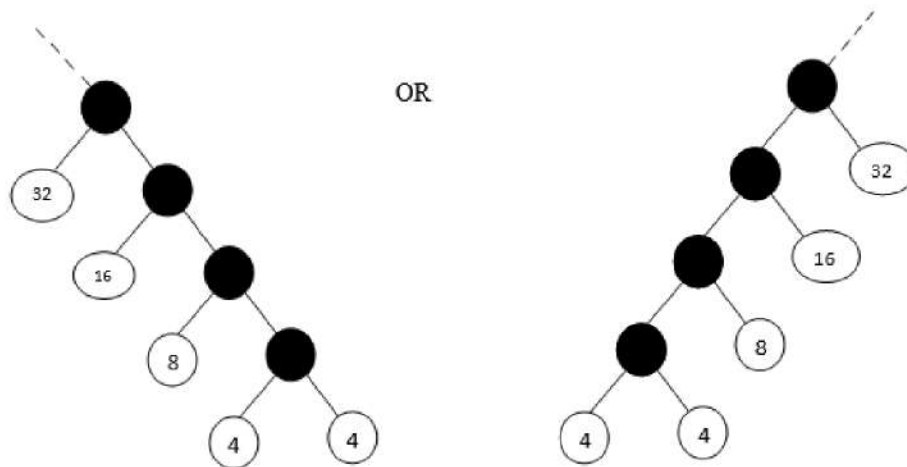
Q(HUF): Given the frequency series for a Huffman as follows:

$$f(i) = \begin{cases} 4, & i = 1 \\ 2^i, & i > 1 \end{cases}$$

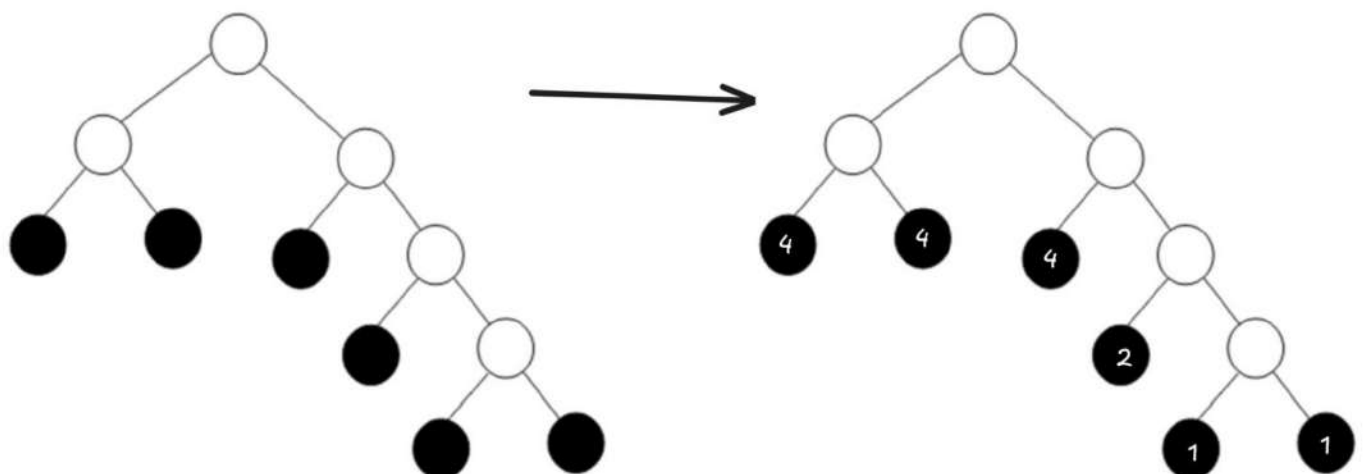
Draw the structure of the Huffman Tree that describes this series.

Solution:

From substitution we get: 4, 4, 8, 16, 32, 64, 128, ...

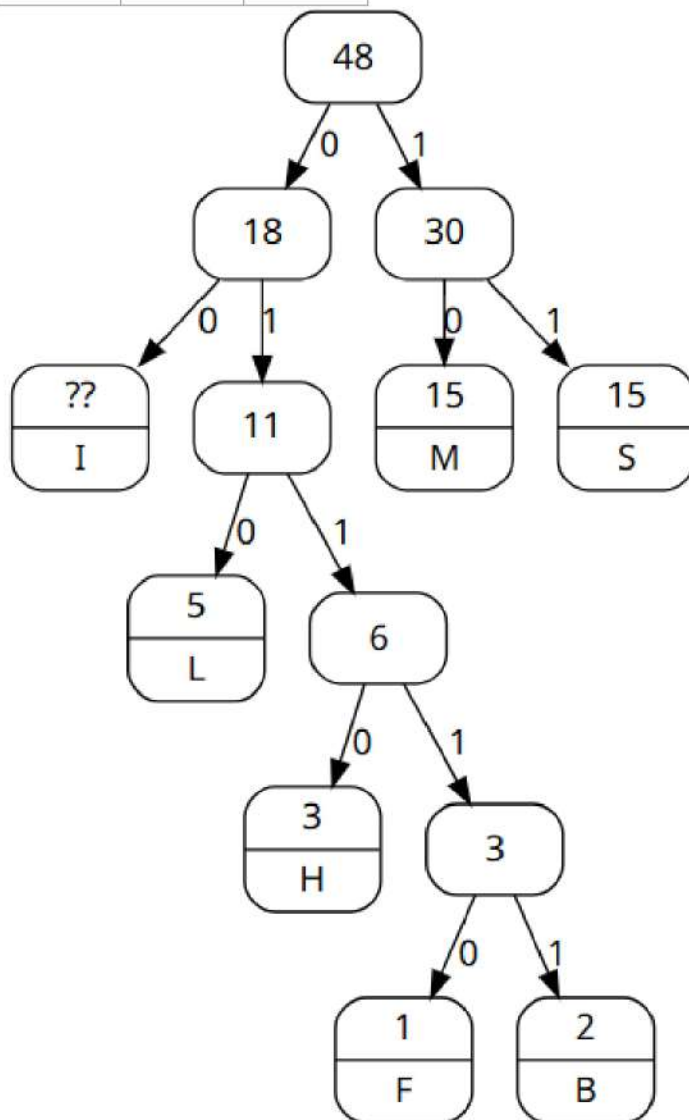


Write a frequency list that the Huffman Code of this frequency Would deterministically create the following structure:



Q(HUF): Find the value of frequency (??) of character I.

Char	Freq.	Code
B	2	01111
F	1	01110
H	3	0110
I	??	00
L	5	010
M	15	10
S	15	11

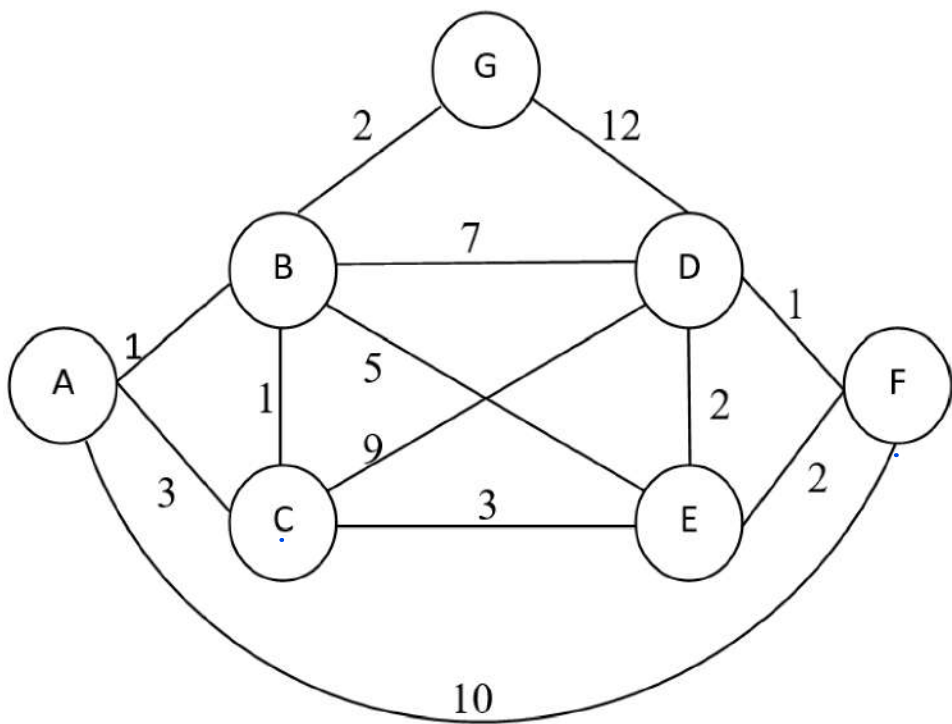


Solution:

$$7 \leq ?? \leq 11$$

□ Graphs

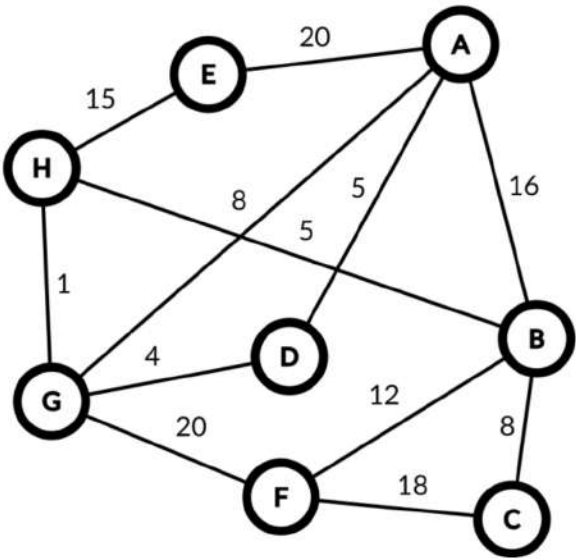
Q(GRAPH): Consider the following undirected, weighted graph, Step through Dijkstra’s algorithm to calculate the single-source shortest paths from A to every other vertex. Show your steps in the table below. Cross out old values and write in new ones, from left to right within each cell, as the algorithm proceeds. Also, list vertices in the order, which you marked them known. Finally, indicate the lowest-cost path from node A to node F.



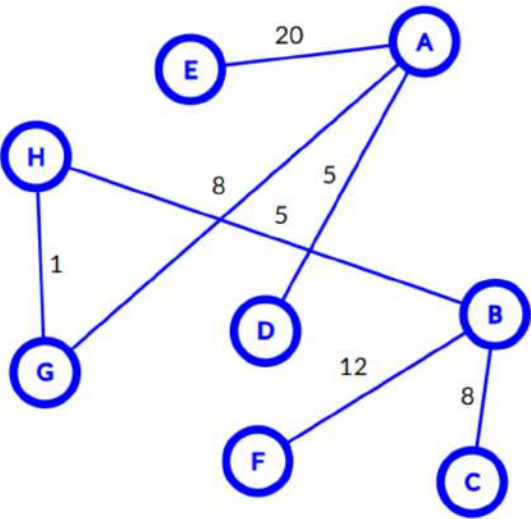
Vertex	known	Cost	Path
A	Yes	0	0
B	Yes	1	A
C	Yes	3 2	A- B
D	Yes	∞ 8 7	B, E
E	Yes	∞ 6 5	B- C
F	Yes	10 7	A, E
G	Yes	∞ 3	B

Lowest-Cost path from A to F: A → B → C → E → F

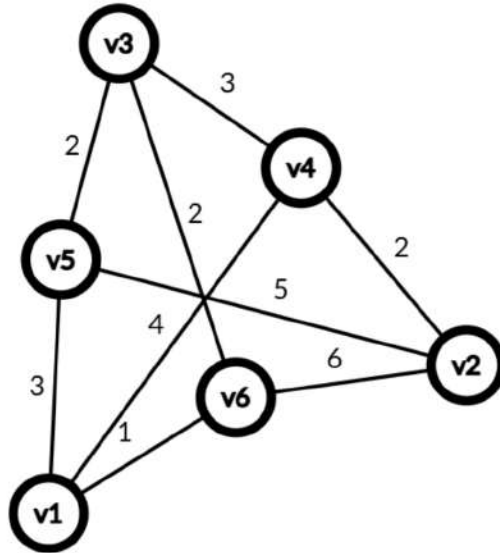
Q(GRAPH): Trace prim’s algorithm for the following graph. Take vertex A as start point.



V	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV
A	No	0	-	Yes	0	-	Yes	0	-	Yes	0	-	Yes	0	-	Yes	0	-	Yes	0	-
B	No	∞	-	No	16	A	No	16	A	No	16	A	No	14	H	Yes	14	H	Yes	14	H
C	No	∞	-	No	∞	-	No	∞	-	No	∞	-	No	∞	-	No	24	B	Yes	24	B
D	No	∞	-	No	5	A	Yes	5	A	Yes	5	A	Yes	5	A	Yes	5	A	Yes	5	A
E	No	∞	-	No	20	A	No	20	A	No	20	A	No	20	A	No	20	A	Yes	20	A
F	No	∞	-	No	∞	-	No	∞	-	No	28	G	No	28	G	No	26	B	Yes	26	B
G	No	∞	-	No	8	A	No	8	A	Yes	8	A	Yes	8	A	Yes	8	A	Yes	8	A
H	No	∞	-	No	∞	-	No	∞	-	No	9	G	Yes	9	G	Yes	9	G	Yes	9	G

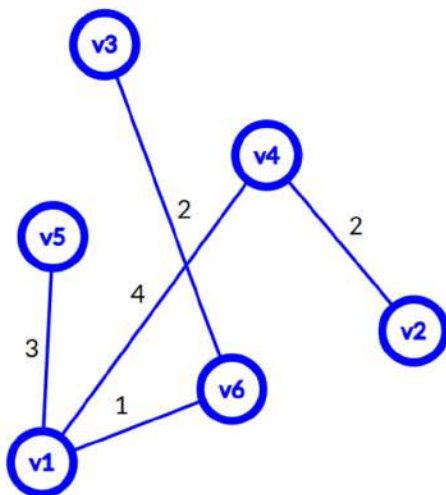


Q(GRAPH): Use Prim's algorithm to find a minimum spanning tree of the graph in Figure below with v1 as the root. What is the time complexity of Prim's algorithm? Note that you should present your answer step by step.



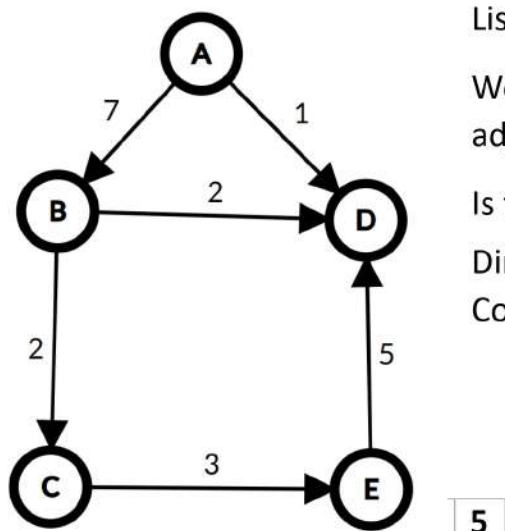
V	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV	K	DV	PV
V1	No	0	-	Yes	0	-	Yes	0	-	Yes	0	-	Yes	0	-	Yes	0	-	Yes	0	-
V2	No	∞	-	No	∞	-	No	7	V6	No	7	V6	No	7	V6	No	6	V4	Yes	6	V4
V3	No	∞	-	No	∞	-	No	3	V6	Yes	3	V6	Yes	3	V6	Yes	3	V6	Yes	3	V6
V4	No	∞	-	No	4	V1	No	4	V1	No	4	V1	No	4	V1	Yes	4	V1	Yes	4	V1
V5	No	∞	-	No	3	V1	No	3	V1	No	3	V1	Yes	3	V1	Yes	3	V1	Yes	3	V1
V6	No	∞	-	No	1	V1	Yes	1	V1	Yes	1	V1	Yes	1	V1	Yes	1	V1	Yes	1	V1

Solution:



Prim's algorithm has a time complexity of:
 $O(V^2)$: with the simple implementation and $O((V + E) \log V)$: with a binary heap.

Q(GRAPH): For the following graph answer the following:



List 2 valid Topological Sort ordering or one.

Worst-case Big-O for Topological Sort as adjacency list (with queue).

Is the graph? (you can choose more than 1)

Directed, weakly connected, Cyclic, Undirected, Complete, strongly connected, Acyclic

1.

		5			
A	0	-	-	-	-
B	1	0	-	-	-
C	1	1	0	-	-
D	3	2	1	1	0
E	1	1	1	0	-
Enqueue	A	B	C	E	D
Dequeue	A	B	C	E	D

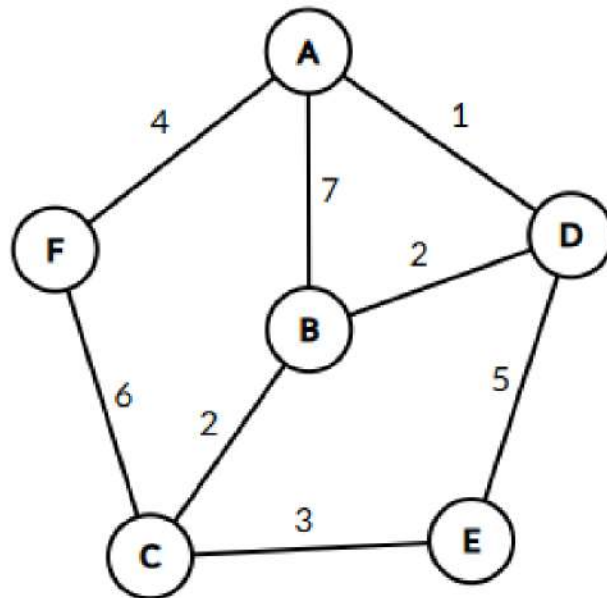
There is only one valid Topological Sort:

A → B → C → E → D

2. $O(V + E)$

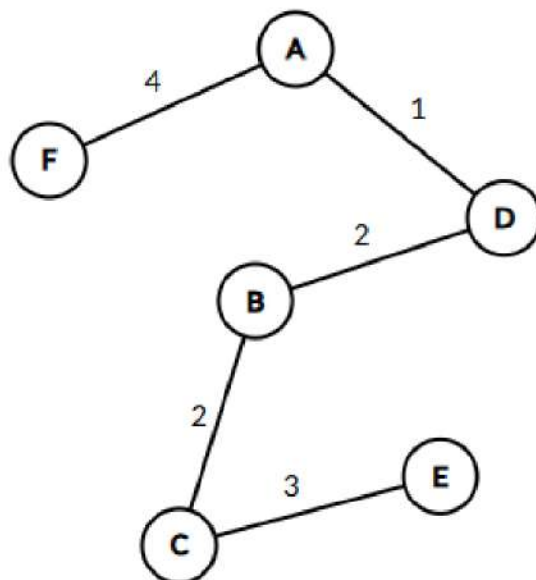
3. Directed, weakly connected, and Acyclic.

Q(GRAPH): Use Kruskal's Algorithm to find a minimum spanning tree of the graph in Figure below with A as the root. What is the time complexity of Kruskal's algorithm? Note that you should present your answer step by step.



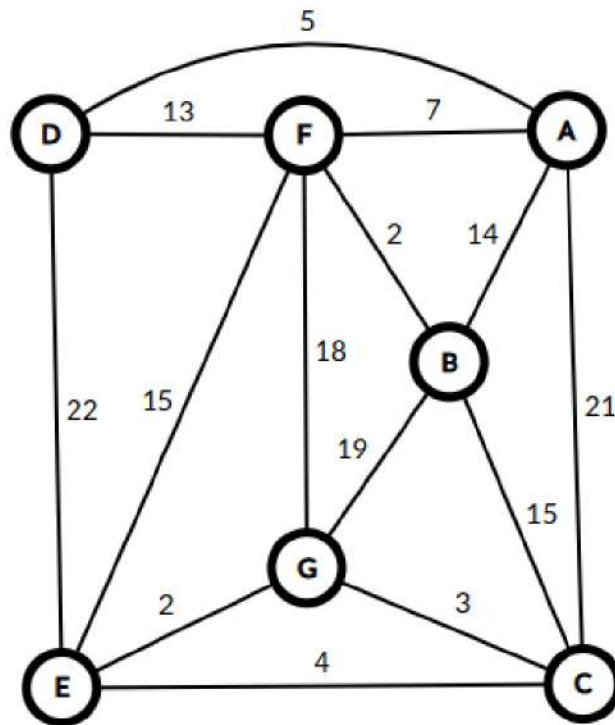
First sort edges:

Edge	Cost	Action
(A, D)	1	Accept
(B, C)	2	Accept
(B, D)	2	Accept
(C, E)	3	Accept
(F, A)	4	Accept
(E, D)	5	Reject
(C, F)	6	Reject
(A, B)	7	Reject

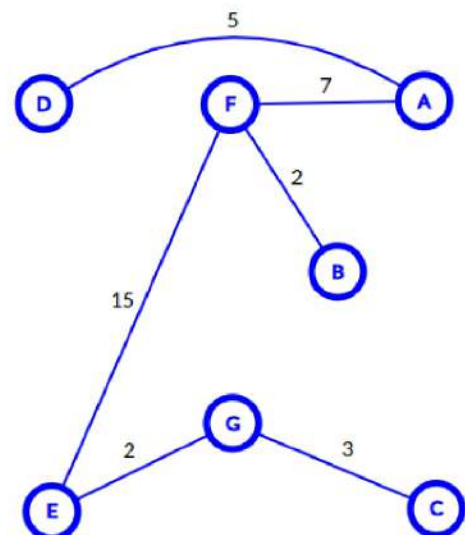


$O(E \log E)$

Q(GRAPH): Step through Kruskal's algorithm to calculate a minimum spanning tree of the graph. Show your steps in the table below, including the disjoint sets at each iteration. If you can select two edges with the same weight, select the edge that would come alphabetically last (e.g., select E–F before B–C. Also, select A–F before A–B).



Edge	Cost	Total	Disjoint Sets	Action
(E, G)	2	2	(EG)(A)(B)(C)(D)(F)	Accept
(B, F)	2	4	(EG)(BF)(A)(C)(D)	Accept
(C, G)	3	7	(CEG)(BF)(A)(D)	Accept
(E, C)	4	-	-	Reject
(A, D)	5	12	(AD)(BF)(CEG)	Accept
(A, F)	7	19	(ABDF)(CEG)	Accept
(D, F)	13	-	-	Reject
(A, B)	14	-	-	Reject
(E, F)	15	34	(ABCDEFG)	Accept
-	-	-	-	Reject Rest



❑ Backtracking

The best way to implement backtracking is through recursion, and all backtracking code can be summarized as per the given Pseudocode:

```
void SOLVE(parameters) {  
    if (valid solution) {  
        store the solution;  
        return;  
    }  
    for (all choice) {  
        if (valid choice) {  
            APPLY(choice);  
            SOLVE(parameters);  
            BACKTRACK(remove choice);  
        }  
    }  
    return;  
}
```

»Important problems you need to solve: «

Search for D: [Top 20 Backtracking Algorithm Interview Questions - GeeksforGeeks](https://www.geeksforgeeks.org/top-20-backtracking-algorithm-interview-questions/)

(<https://www.geeksforgeeks.org/top-20-backtracking-algorithm-interview-questions/>)

Q(Horners): Write the best algorithm to solve the following equation:

$$f(x) = x^{12} + 7x^8 + 5x^5 + 12x^2 + 9$$

Solution:

$f(x)$ is Sparse so best to use Sparse representation

$e = [12, 1, 8, 7, 5, 5, 2, 12, 0, 9]$

```
solve(e, x) {
  n = len(e)
  sum = 0
  for i from 0 to n - 1 begin
    sum = (sum + a[i]) * x ↑ (e[i] - e[i+1])
  end for

  return sum
}
```

result in: $x^2(x^3(x^3(x^4 + 7) + 5) + 12) + 9$

Q(Horners): Write the best algorithm to solve the following equation:

$$f(x) = 8x^5 + 3x^4 + 2x^3 + 6x^2 + 7x + 4$$

Solution:

$f(x)$ is Dense so best to use Dense representation

$e = [5, 8, 3, 2, 6, 7, 4]$

```
solve(e, x) {
  n = len(e) - 1
  i = 2
  sum = e[1]
  for i from 2 to n begin
    sum = sum * x + a[i]
  endfor
  return sum
}
```

result in: $x(x(x(x(8x + 3) + 2) + 6) + 7) + 4$

□ Parallel Processing

Q(PP): Trace the below algorithm considering the given list of numbers. Please note that this algorithm is designed for 1D mesh parallel processing architecture. Please show all the steps.

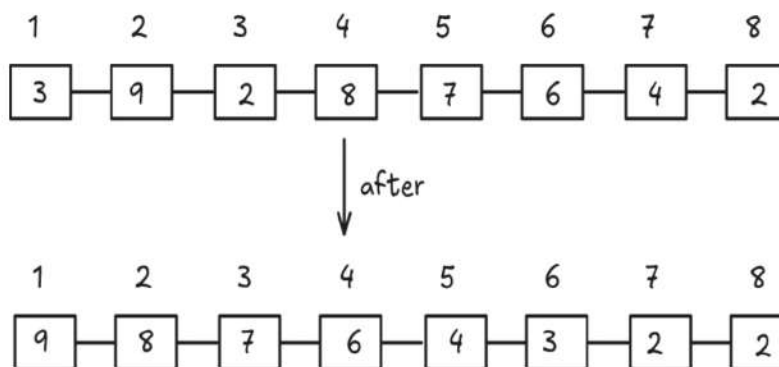
Numbers = [3, 9, 2, 8, 7, 6, 4, 2]

n = 8

```

Procedure testCode(n) begin
  for i from 1 to n do
    if i is odd then
      for j from 0 to n/2-1 do
        if A2j+1 < A2j+2 then
          exchange (A2j+1, A2j+2)
        endif
      endfor
    endif
    if i is even then
      for j from 1 to n/2-1 do
        if A2j < A2j+1 then
          exchange (A2j, A2j+1)
        endif
      endfor
    endif
  endfor
end

```



i	j	1	2	3	4	5	6	7	8
1	0	9	3	2	8	7	6	4	2
	1	9	3	8	2	7	6	4	2
	2	9	3	8	2	7	6	4	2
	3	9	3	8	2	7	6	4	2
2	1	9	8	3	2	7	6	4	2
	2	9	8	3	7	2	6	4	2
	3	9	8	3	7	2	6	4	2
3	0	9	8	3	7	2	6	4	2
	1	9	8	7	3	2	6	4	2
	2	9	8	7	3	6	2	4	2
	3	9	8	7	3	6	2	4	2
4	1	9	8	7	3	6	2	4	2
	2	9	8	7	6	3	2	4	2
	3	9	8	7	6	3	4	2	2
5	0	9	8	7	6	3	4	2	2
	1	9	8	7	6	3	4	2	2
	2	9	8	7	6	4	3	2	2
	3	9	8	7	6	4	3	2	2
6	1	..							
	2	..							

Q(PP): Answer the following:

All computer architectures can be classified into four categories. Describe these categories in details.

1. **Single Instruction stream Single Data stream (SISD)**: This is the architecture used by sequential computers, where one instruction is applied to one data set at a time. It operates by executing instructions on data sequentially.
2. **Multiple Instruction stream Single Data stream (MISD)**: This architecture is not commonly used in computers. It involves applying multiple instructions to the same data at the same time. An example of this is performing addition, multiplication, subtraction, and division on the same two numbers.
3. **Single Instruction stream Multiple Data stream (SIMD)**: In this architecture, multiple processors operate under the control of a single instruction stream, with each processor having its own private memory to store data. During a given time unit, a selected number of processors execute the same instruction on different sets of data.
4. **Multiple Instruction stream Multiple Data stream (MIMD)**: This architecture is similar to SIMD, but uses multiple instruction streams instead of one. Each processor executes instructions from an instruction stream on a set of data, working independently and synchronously.

What we mean by degree of parallelism?

The degree of parallelism refers to the **number of processors** in a multiprocessor system, indicating the **number of tasks** that can be carried out simultaneously.

Parallel processing topology is the way by which the processors in parallel computers are connected. Describe in details the Perfect Shuffle and Shuffle Exchange topology and mention the different between them.

- **Perfect Shuffle** topology connects processors in parallel computing. For a system with 2^n processors, each processor (i) connects to processor (j) based on these rules:

$$j = \begin{cases} i & , i = 0 \text{ or } i = n - 1 \\ 2 \times i \% (n - 1) & , \text{otherwise} \end{cases} \quad \text{OR} \quad j = \begin{cases} 2^i & , 0 \leq i \leq \frac{n}{2} \\ 2^i + 1 - n & , \frac{n}{2} \leq i \leq n \end{cases}$$

- **Shuffle-Exchange** topology connects processors in parallel computing, which is similar to the Perfect Shuffle topology but includes additional connections.

The key difference is the addition of **bidirectional connections** between each even-numbered processor and its immediate successor. These are known as exchange connections.

Q(PP): Perform a Bitonic sort of the following list of numbers using SIMD-CC model, into ascending order.

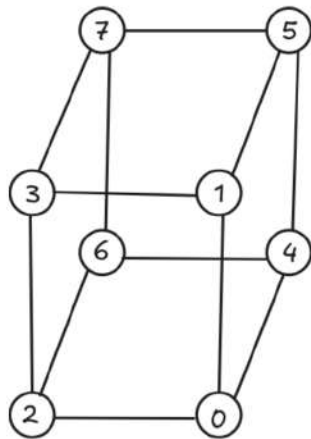
Numbers = [9, 2, 16, 27, 5, 3, 70, 33]

```

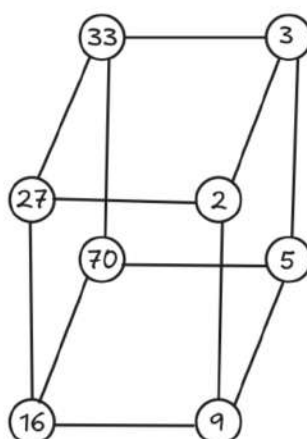
BitonicMergeSort(SIMD-CC) Begin
  for i = 0 to m-1 Do
    for j = i down to 0 Do
      for j = i down to 0 Do
        d = 2j
        for all pk, where 0 ≤ k ≤ 2m - 1 Do
          if (k Mod 2d < d) then
            tk = ak+d
            if (k Mod 2i+2 < 2i+1) then
              bk = max(tk, ak)
              ak = min(tk, ak)
            else
              bk = min(tk, ak)
              ak = max(tk, ak)
            end if
          end if
        end for
      end for
    end for
  end for
end

```

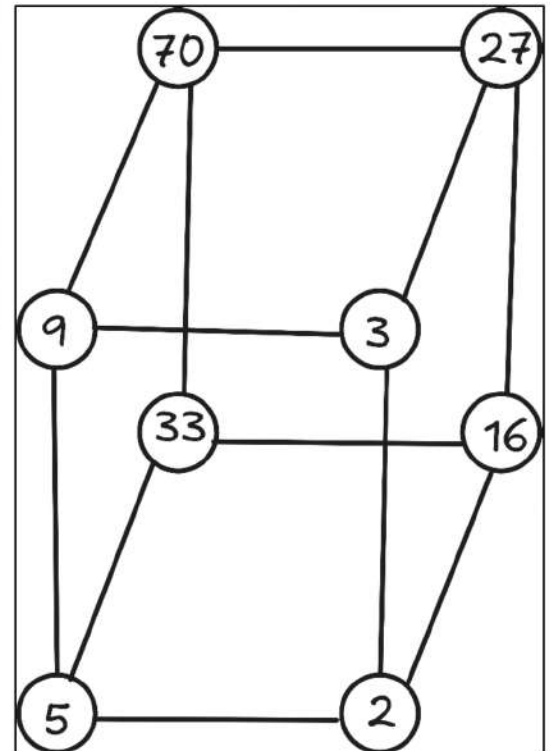
i	j	d	0	1	2	3	4	5	6	7
0	0	1	2	x	27	x	3	x	70	x
1	1	2	2	9	x	x	70	33	x	x
	0	1	2	x	16	x	70	x	5	x
2	2	4	2	9	16	27	x	x	x	x
	1	2	2	3	x	x	16	27	x	x
	0	1	2	x	5	x	16	x	33	x



Processors



Data



Q(PP): Perform a Bitonic sort of the following list of numbers using SIMD-CC model, into descending order.

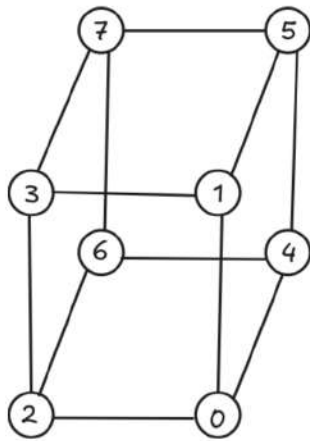
Numbers = [9, 2, 16, 27, 5, 3, 70, 33]

```

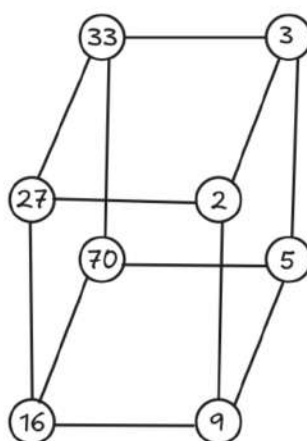
BitonicMergeSort(SIMD-CC) Begin
  for i = 0 to m-1 Do
    for j = i down to 0 Do
      for j = i down to 0 Do
        d = 2j
        for all pk, where 0 ≤ k ≤ 2m - 1 Do
          if (k Mod 2d < d) then
            tk = ak+d
            if (k Mod 2i+2 < 2i+1) then
              bk = min(tk, ak)
              ak = max(tk, ak)
            else
              bk = max(tk, ak)
              ak = min(tk, ak)
            end if
          end if
        end for
      end for
    end for
  end for
end

```

i	j	d	0	1	2	3	4	5	6	7
0	0	1	2	x	16	x	3	x	70	x
1	1	2	16	27	x	x	3	5	x	x
	0	1	27	x	9	x	3	x	33	x
2	2	4	27	16	33	70	x	x	x	x
	1	2	33	70	x	x	9	5	x	x
	0	1	70	x	27	x	9	x	3	x



Processors



Data

